

The 40-Pattern DSA Decoder

Recognition-first DSA — plus the data structures that power real systems.

5 chapters · compiled from the author's engineering notes and essays

By Pradyumna Chippigiri

Compiled 4 October 2026

WHAT'S INSIDE

- The 40-Pattern DSA Decoder
- Skip List
- Inverted Index
- Time-series delta encoding
- Database Indexing Best Practises

Skip List	10
<i>Recognition-first pattern catalogue</i>	
The 40-Pattern DSA Decoder	3

CONTENTS

The 40-Pattern DSA Decoder

2026 · Recognition-first pattern catalogue

Source: Authored for Beyond The Interviews

Recognition-first DSA. Stop memorizing solutions; learn the 40 doors.

How to use this decoder

- Read a pattern's CUE before its solution. In interviews, 60% of the battle is naming the pattern in under 2 minutes.
- After every problem you solve, write one sentence: the pattern and why it applied. If you cannot, you did not learn it.
- Spaced repetition: re-solve each model problem on Day 3, Day 7, Day 21. Fail a review = restart the interval.
- Constraints are the cheapest filter: $n \leq 20$ means backtracking/bitmask is fine; $n \leq 100000$ means you need $O(n \log n)$ or better; $O(n^2)$ is dead on arrival.

Family: Windows and Pointers

1. Fixed-Size Sliding Window

CUE: "subarray/substring of size k", max/avg over every k-block. CORE: add entering element, remove leaving element in $O(1)$. PROBLEMS: LC 643 Maximum Average Subarray I, LC 567 Permutation in String, LC 438 Find All Anagrams. COMPLEXITY: $O(n)$ time, $O(1)$ space. TRAP: window size is $r-l+1$; forgetting to subtract the element leaving the window.

2. Variable-Size Sliding Window

CUE: "longest/shortest contiguous... with at most/exactly k distinct..." CORE: expand right always; shrink left while invalid. Works only when validity is monotone. PROBLEMS: LC 3 Longest Substring Without Repeating, LC 76 Minimum Window Substring, LC 904 Fruit Into Baskets. COMPLEXITY: $O(n)$ time, $O(k)$ space. TRAP: shrinking with if instead of while; using windows with negative numbers (monotonicity breaks - go Prefix Sum).

3. Two Pointers (converging)

CUE: sorted array + pair/triplet/target sum. CORE: move ends inward based on comparison against target. PROBLEMS: LC 167 Two Sum II, LC 15 3Sum, LC 11 Container With Most Water. COMPLEXITY: $O(n \log n)$ with sort, then $O(n)$. TRAP: not skipping duplicates in 3Sum.

4. Fast-Slow Pointers

CUE: linked list cycle, middle node, or "duplicate number in array of $1..n$ ". CORE: Floyd's tortoise-hare; on meeting, reset one pointer to head, advance both by 1 to find cycle start. PROBLEMS: LC 141, LC 142 Linked List Cycle II, LC 287 Find the Duplicate, LC 876 Middle. COMPLEXITY: $O(n)$ time, $O(1)$ space. TRAP: finding the meeting point but botching the restart phase.

5. Prefix Sum + HashMap

CUE: "count subarrays with sum k", range sums with negatives. CORE: $\text{prefix}[i] - \text{prefix}[j] = k$ means $\text{prefix}[j] = \text{prefix}[i] - k$; store counts of seen prefixes. PROBLEMS: LC 560 Subarray Sum Equals K, LC 974 Subarray Sums Divisible by K, LC 303 Range Sum. COMPLEXITY: $O(n)$ time, $O(n)$ space. TRAP: not seeding the map with $\{0:1\}$ before the loop.

6. Hashing: Seen-Before and Grouping

CUE: "have I seen this?", frequency counts, group-by canonical key. CORE: trade $O(n)$ space for $O(1)$ lookups; normalize keys (sorted signature for anagrams). PROBLEMS: LC 1 Two Sum, LC 49 Group Anagrams, LC 128 Longest Consecutive Sequence. COMPLEXITY: $O(n)$ time, $O(n)$ space. TRAP: LC 128 - scan only from sequence starts (x-1 not in set), not from every element.

Family: Stacks and Queues

7. Matching / Expression Stack

CUE: brackets, undo, evaluate postfix/infix expressions. CORE: push unresolved tokens; pop when the current token resolves an earlier one. PROBLEMS: LC 20 Valid Parentheses, LC 150 Evaluate RPN, LC 227 Basic Calculator II. COMPLEXITY: $O(n)$ time, $O(n)$ space. TRAP: operator precedence - apply $*$ and $/$ immediately, defer $+$ and $-$ with a sign.

8. Monotonic Stack

CUE: "next greater/smaller element", histogram, stock span, visible buildings. CORE: keep the stack sorted by value; pop everything smaller when a new element arrives. Each index pushed once. PROBLEMS: LC 739 Daily Temperatures, LC 496 Next Greater Element, LC 84 Largest Rectangle. COMPLEXITY: $O(n)$ time, $O(n)$ space. TRAP: store indices, not values; wrong strictness ($<$ vs $<=$) with duplicates.

9. Monotonic Deque

CUE: max/min of every sliding window. CORE: deque of indices with decreasing values; pop front when expired, pop back when smaller. PROBLEMS: LC 239 Sliding Window Maximum, LC 1438 Longest Continuous Subarray With Abs Diff. COMPLEXITY: $O(n)$ time, $O(k)$ space. TRAP: leaving expired indices at the front.

10. Queue/Stack via Design

CUE: "implement X using Y", recent-counter, circular buffer. CORE: amortized transfer between two stacks only when the out-stack is empty. PROBLEMS: LC 232 My Queue, LC 225 My Stack, LC 933 Recent Calls. COMPLEXITY: amortized $O(1)$. TRAP: shuffling elements on every operation instead of on empty.

Family: Binary Search

11. Boundary Binary Search

CUE: sorted input + "first/last index where predicate true". CORE: keep the invariant that the answer lies in $[lo, hi]$; shrink strictly every branch. PROBLEMS: LC 704 Binary Search, LC 34 First and Last Position, LC 35 Search Insert. COMPLEXITY: $O(\log n)$ time, $O(1)$ space. TRAP: mixing templates ($<$ vs $<=$) causes infinite loops; overflow with $(lo+hi)/2$ in Java/C++.

12. Rotated Array Search

CUE: sorted-then-rotated; find min or target. CORE: one half is always sorted - identify it, then check whether the target lies inside it. PROBLEMS: LC 33 Search in Rotated Array, LC 153 Find Minimum, LC 81 with duplicates. COMPLEXITY: $O(\log n)$ time, $O(1)$ space. TRAP: duplicates (LC 81) need $lo++/hi--$ shrink.

13. Binary Search on Answer

CUE: "minimize the maximum / maximize the minimum", answer range up to 10^9 . CORE: a monotone feasibility predicate; binary-search the answer space, greedy $O(n)$ check per candidate. PROBLEMS: LC 875 Koko Eating Bananas, LC 1011 Capacity to Ship, LC 410 Split Array Largest. COMPLEXITY: $O(n \log R)$ time, $O(1)$ space. TRAP: bad bounds ($lo = \max(\text{nums})$, $hi = \sum(\text{nums})$) and ceiling division errors.

Family: Graphs and Grids

14. Grid Flood Fill (BFS/DFS)

CUE: islands, regions, connected cells. CORE: 4-directional traversal with visited marking. PROBLEMS: LC 200 Number of Islands, LC 733 Flood Fill, LC 695 Max Area of Island. COMPLEXITY: $O(mn)$ time and space. TRAP: enqueueing without marking visited - duplicates blow the queue.

15. Shortest Path BFS

CUE: unweighted graph + "minimum steps/moves/turns". CORE: BFS explores in level order; the first time you reach the target is optimal. PROBLEMS: LC 127 Word Ladder, LC 752 Open the Lock, LC 365 Water Jug. COMPLEXITY: $O(V+E)$ time, $O(V)$ space. TRAP: incomplete neighbor generation (all 8 lock dials, all 26 letters).

16. Multi-Source BFS

CUE: distance from ALL gates/zeros/rotten oranges simultaneously. CORE: seed the queue with every source at time 0; one sweep. PROBLEMS: LC 542 01 Matrix, LC 1162 As Far From Land, LC 994 Rotting Oranges. COMPLEXITY: $O(mn)$ time and space. TRAP: BFS from each cell separately - $O((mn)^2)$ disaster.

17. Subsets Backtracking

CUE: "all combinations/subsets/power set". CORE: include/exclude each element, or iterate from index i ; copy the path at every node (subsets) or leaf (combinations). PROBLEMS: LC 78 Subsets, LC 90 Subsets II, LC 39 Combination Sum. COMPLEXITY: $O(2^n * n)$ time, $O(n)$ space. TRAP: Subsets II duplicates - sort, then skip equal siblings at the same level.

18. Permutations Backtracking

CUE: "all arrangements", order matters. CORE: `used[]` array (or swap-in-place), recurse, undo. PROBLEMS: LC 46 Permutations, LC 47 Permutations II, LC 77 Combinations. COMPLEXITY: $O(n * n!)$ time, $O(n)$ space. TRAP: forgetting to unset `used[]` after the recursive call.

19. Grid/String DFS with Pruning

CUE: word search, palindrome partitioning, board + dictionary. CORE: recurse cell by cell; prune when no trie prefix survives; restore state on exit. PROBLEMS: LC 79 Word Search, LC 212 Word Search II, LC 131 Palindrome Partitioning. COMPLEXITY: $O(mn * 4^L)$ worst case. TRAP: not unmarking the visited cell after

recursion; no trie in LC 212 = TLE.

20. Topological Sort (Kahn's)

CUE: prerequisites, "valid ordering", build/install order. CORE: repeatedly emit indegree-0 nodes and decrement successors. If emitted count $< V$, there is a cycle. PROBLEMS: LC 207 Course Schedule, LC 210 Course Schedule II, LC 269 Alien Dictionary. COMPLEXITY: $O(V+E)$ time, $O(V)$ space. TRAP: inverted edge direction ([a,b] means b before a).

21. Union-Find

CUE: dynamic connectivity, counting components, redundant edge, merging accounts. CORE: parent array + find with path compression + union by rank; decrement component count on successful merges. PROBLEMS: LC 547 Friend Circles, LC 684 Redundant Connection, LC 721 Accounts Merge. COMPLEXITY: near $O(1)$ amortized per op. TRAP: skipping path compression/rank - degenerates to $O(n)$ chains.

Family: Heaps and Ordering

22. Top-K Heap

CUE: "kth largest / k closest / k most frequent". CORE: maintain a size-k MIN-heap when hunting the k largest; the root is the answer. PROBLEMS: LC 215 Kth Largest, LC 347 Top K Frequent, LC 973 K Closest Points. COMPLEXITY: $O(n \log k)$ time, $O(k)$ space. TRAP: wrong heap orientation - k largest needs a min-heap.

23. Two Heaps

CUE: running median, streaming quantiles. CORE: max-heap for the lower half, min-heap for the upper; rebalance so sizes differ by at most 1. PROBLEMS: LC 295 Find Median from Stream, LC 502 IPO, LC 480 Sliding Median. COMPLEXITY: $O(\log n)$ insert, $O(1)$ median. TRAP: skipping the push-move-rebalance rotation.

24. K-Way Merge

CUE: k sorted lists/files/streams merged into one. CORE: heap holds one cursor per source; pop the min, push that source's next element. PROBLEMS: LC 23 Merge k Lists, LC 373 K Pairs Smallest Sums, LC 632 Smallest Range. COMPLEXITY: $O(N \log k)$ time, $O(k)$ space. TRAP: heap tuples need a tiebreaker index or comparisons crash.

Family: Intervals and Greedy

25. Merge/Insert Intervals

CUE: [start, end] pairs + merge/consolidate/insert. CORE: sort by start; overlap iff $\text{cur.start} \leq \text{last.end}$; extend with $\max()$. PROBLEMS: LC 56 Merge Intervals, LC 57 Insert Interval, LC 452 Minimum Arrows. COMPLEXITY: $O(n \log n)$ time. TRAP: assigning $\text{last.end} = \text{end}$ instead of $\max(\text{last.end}, \text{end})$ - contained intervals truncate silently.

26. Activity Selection Greedy

CUE: "minimum removals / maximum compatible meetings". CORE: sort by END time; greedily keep the earliest-finishing compatible interval. PROBLEMS: LC 435 Non-overlapping Intervals, LC 452, LC 252 Meeting Rooms. COMPLEXITY: $O(n \log n)$ time. TRAP: sorting by start (merge-intervals reflex) gives wrong answers.

27. Min Rooms / Sweep Line

CUE: "how many rooms/resources are needed concurrently?" CORE: min-heap of end times, or +1/-1 event sweep (end processed before start on ties). PROBLEMS: LC 253 Meeting Rooms II, LC 1094 Car Pooling, LC 1851. COMPLEXITY: $O(n \log n)$ time. TRAP: tie-breaking - back-to-back meetings need the room freed first.

28. Jump-Game Greedy

CUE: maximize reach with local choices. CORE: track the farthest reachable index; justify with an exchange argument. PROBLEMS: LC 55 Jump Game, LC 45 Jump Game II, LC 406 Queue Reconstruction. COMPLEXITY: $O(n)$ time, $O(1)$ space. TRAP: asserting greedy without being able to say why it is safe.

Family: Dynamic Programming

29. 1D Linear DP

CUE: $dp[i]$ depends on $dp[i-1]$, $dp[i-2]$; take-or-skip per index. CORE: define the state precisely, roll two variables. PROBLEMS: LC 70 Climbing Stairs, LC 198 House Robber, LC 91 Decode Ways. COMPLEXITY: $O(n)$ time, $O(1)$ space. TRAP: reaching for greedy on House Robber; botched base cases.

30. 2D Grid DP

CUE: paths through a grid, min path sum, obstacles. CORE: $dp[i][j] = f(\text{up}, \text{left})$. PROBLEMS: LC 62 Unique Paths, LC 64 Min Path Sum, LC 63 with obstacles. COMPLEXITY: $O(mn)$ time, $O(n)$ rolled space. TRAP: first row/column initialization when obstacles block them.

31. 0/1 Knapsack

CUE: pick each item at most once to hit a capacity/target; "partition into equal subsets". CORE: $dp[w]$ over items, iterate capacity BACKWARD. PROBLEMS: LC 416 Partition Equal Subset, LC 494 Target Sum, LC 1049 Last Stone Weight. COMPLEXITY: $O(n \cdot S)$ time, $O(S)$ space. TRAP: forward capacity loop lets one item be used twice.

32. Unbounded Knapsack

CUE: items reusable - coins making an amount, ribbon cutting. CORE: $dp[a] = \text{best}(dp[a - \text{coin}])$; forward iteration permits reuse. PROBLEMS: LC 322 Coin Change, LC 518 Coin Change II, LC 279 Perfect Squares. COMPLEXITY: $O(A \cdot \text{coins})$ time, $O(A)$ space. TRAP: loop order decides combinations vs permutations (518 vs 377).

33. LCS Family (two sequences)

CUE: two strings/arrays + longest common / edit operations. CORE: dp over prefixes; match moves diagonally, else take max of top/left. PROBLEMS: LC 1143 LCS, LC 72 Edit Distance, LC 718 Repeating Subarray. COMPLEXITY: $O(mn)$ time, $O(\min(m,n))$ space. TRAP: LCS vs Edit Distance transitions (2 moves vs 3 moves on mismatch).

34. String Segmentation DP

CUE: "can the string be segmented", palindromic substrings. CORE: $dp[i]$ over prefixes; check every dictionary word / palindrome ending at i . PROBLEMS: LC 139 Word Break, LC 5 Longest Palindromic Substring, LC 647

Palindromic Substrings. COMPLEXITY: $O(n^2)$ time. TRAP: substring slicing inside loops - hidden $O(n^3)$; Decode Ways treats '0' as standalone.

35. Longest Increasing Subsequence

CUE: longest increasing chain, nesting envelopes, chains. CORE: $O(n^2)$ dp ending at i , or tails array + binary search for $O(n \log n)$. PROBLEMS: LC 300 LIS, LC 354 Russian Doll Envelopes, LC 646 Longest Pair Chain. COMPLEXITY: $O(n \log n)$ achievable. TRAP: assuming tails[] is the actual subsequence - it only has the right length.

Family: Specialized

36. Bit Manipulation

CUE: "every element appears twice except one", XOR tricks, bit counting. CORE: XOR cancels pairs; $n \& (n-1)$ clears the lowest set bit. PROBLEMS: LC 136 Single Number, LC 191 Hamming Weight, LC 338 Counting Bits. COMPLEXITY: $O(n)$ time, $O(1)$ space. TRAP: arithmetic right shift on negatives in Java/C++ (use unsigned shift).

37. Trie

CUE: prefix queries, autocomplete, wildcard dictionary, board + word list. CORE: shared-prefix nodes; isWord flag separates word-end from mere prefix. PROBLEMS: LC 208 Implement Trie, LC 211 Wildcard Matching, LC 212 Word Search II. COMPLEXITY: $O(L)$ per operation. TRAP: conflating search vs startsWith semantics.

38. Linked List In-Place Rewiring

CUE: reverse/reorder/rotate a list with $O(1)$ space mandated. CORE: save next, redirect current, advance prev/curr; dummy head simplifies head mutations. PROBLEMS: LC 206 Reverse List, LC 92 Reverse II, LC 143 Reorder List, LC 21 Merge Two Lists. COMPLEXITY: $O(n)$ time, $O(1)$ space. TRAP: overwriting .next before saving it.

39. Matrix Simulation

CUE: spiral order, rotate image in place, set zeroes. CORE: maintain converging layer boundaries; rotation = transpose + reverse rows. PROBLEMS: LC 54 Spiral Matrix, LC 48 Rotate Image, LC 73 Set Matrix Zeroes. COMPLEXITY: $O(mn)$ time. TRAP: boundary drift when one row/column remains.

40. Cyclic Sort

CUE: values form a dense range $1..n$ + find missing/duplicate in $O(1)$ space. CORE: swap each value to its home index; scan for mismatches. PROBLEMS: LC 268 Missing Number, LC 448 Find Disappeared, LC 41 First Missing Positive. COMPLEXITY: $O(n)$ time, $O(1)$ space. TRAP: incrementing i after a swap - the newly arrived value is unchecked; compare before swapping to avoid infinite loops on duplicates.

The 60-second recognition drill

Once a week, pick 5 unseen problems. For each, spend 60 seconds MAX writing: pattern name, the cue that triggered it, and complexity. No coding. This is the exact skill the interview tests first.

Litmus tests worth memorizing

- "Do I need the middle elements?" If no, two pointers; if yes, sliding window.
- "Can I sort without losing information?" If yes, sort first and re-examine.
- Negatives present? Sliding window breaks - think prefix sum + hashmap.
- "Minimize the max / maximize the min"? Binary search on answer.
- Constraints under 20? Backtracking or bitmask is intended.

Skip List

2026-09-14 · distributed systems

Source: Engineering notes — pradyumnac26.github.io

The Skip List data structure isn't very common it's not something we usually come across while learning or solving problems. What motivated me to learn about it was its usage in HNSW-based indexing for finding top-k nearest neighbours, and in indexing for vector databases, etc.

Drawbacks of Linked lists

Linked Lists are dynamic data structures that store values in non-contiguous memory called nodes. Each node refers to the other using memory pointers.

Linked lists are wonderful data structures that let us build without needing to estimate how much memory will be needed upfront.

Arrays let us randomly access any element quickly, but insertion and deletion is slow because elements need to shift to make room or close the gap.

And Linked List came in to solve the weakness of arrays - Linked lists make inserting and deleting fast once we have reached the right spot, but getting to that spot is slow because there's no random access.

Drawbacks of general linked lists:

- Hard to search in less than $O(n)$ time (binary search doesn't work, e.g.)
- Hard to jump to the middle

This is where skip lists come into the picture, as they try to resolve this drawback in Linked List and bring the search time down to $O(\log N)$.

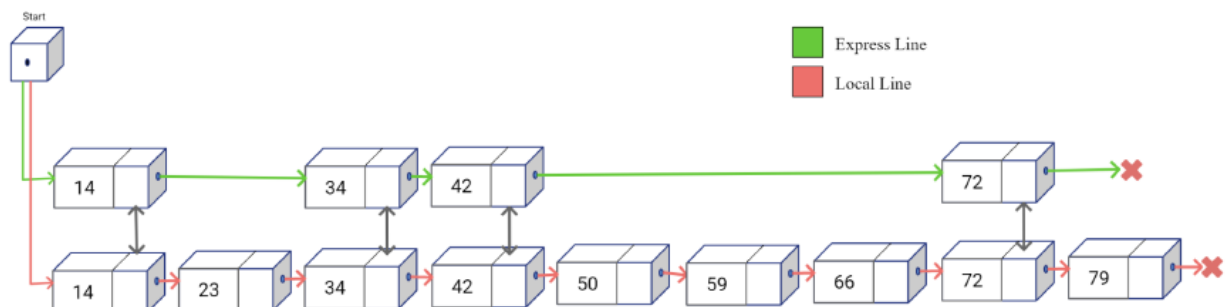
Skip List Data Structure

Skip lists are a probabilistic data structure constructed from several layers of linked lists. It is called as probabilistic data structure because it uses random coin flips to build itself.

The bottom layer is just a regular sorted linked list connecting an ordered sequence of elements.

Each new layer above removes some elements from the underlying layer (based on a fixed probability), producing a sparser version of the layer below.

The diagram below shows a skip list.



How Search works in a Skip List

The search begins at the top layer and moves horizontally along its linked list. If the target element is found there, the search stops immediately and returns, this is exactly why the sparse top layer is useful: finding the element early, at a sparser level, means fewer nodes had to be checked, keeping the time complexity low.

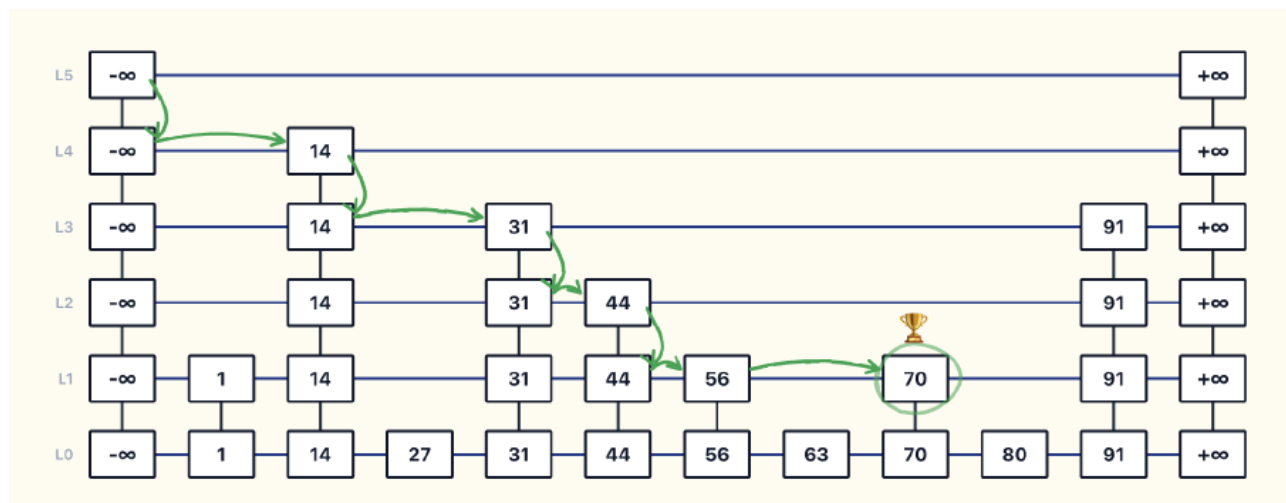
Searching in a skip list happens like this:

When searching for k :

- If $k = \text{key}$, done!
- If $k < \text{next key}$, go down a level.
- If $k \geq \text{next key}$, go right.

This is what gives skip lists their efficiency: Skip lists offer $O(\log n)$ average complexity for both search and insertion/deletion.

Example : Finding 70.



How Insertion works in a Skip List

How does the insertion work in a skip list? Let's understand this even better.

Search-in-skiplist.mov (video — see the online note)

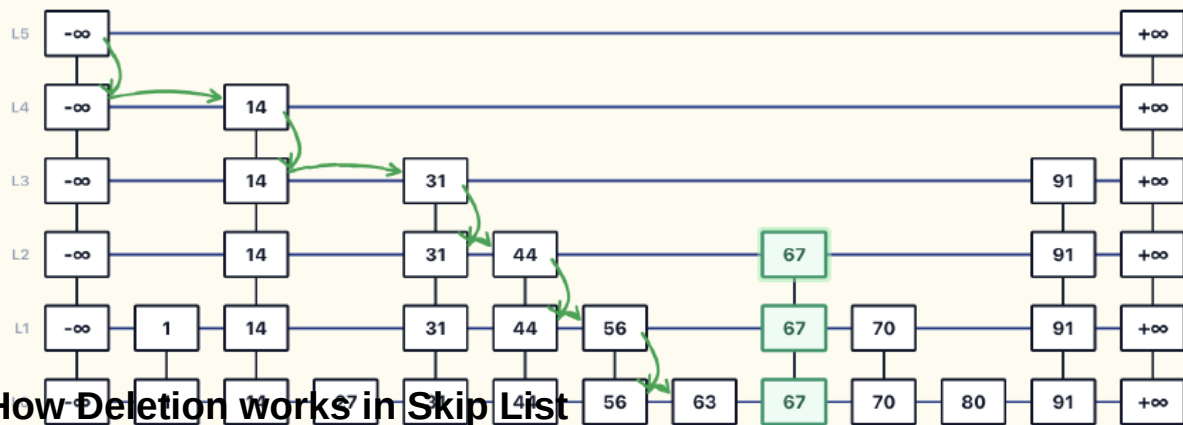
To insert a new key, we first need to find where it should go in the bottom layer of linked list and for this, we just reuse the same search logic we used before. As we apply the search logic and traverse we reach at a position in bottom list where we can't go more to the right, and that's where we insert the new value on the right side of that position

While doing this search, we keep track of the last node visited at each level before as these are the predecessor nodes that'll point to our new node once inserted.

Once we insert the element in Level 0, how do we decide if it should be inserted in other levels or no ?

The answer is that we flip a fair coin ($p = 1/2$) to decide if the node should be promoted one level up, or not. HEADS promotes it one level higher; TAILS stops the promotion. So HEADS then TAILS means the node reaches Level 1. This coin-flip decides the height randomly, which is why skip lists are called probabilistic

Example : Inserting 67

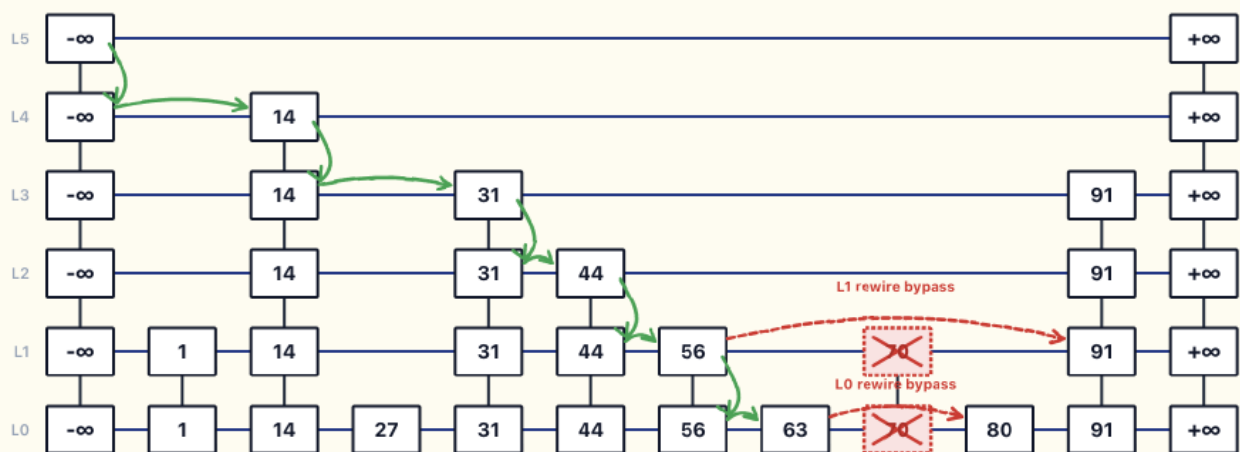


How Deletion works in Skip List

Inserting-in-skiplist.mov (video — see the online note)

Deletion operation in Skip List is pretty straightforward. We first perform search operation to find the location of the node. If we find the node, we simply delete it at all levels. Before deleting a node, we keep track of its predecessor at each level, so we can later point that predecessor directly to whatever comes after the deleted node

Delete 70 example :



Delete 31 example :

delete-in-skiplist.mov (video — see the online note)

Inverted Index

2026-07-16 · distributed systems

Source: Engineering notes — pradyumnac26.github.io

How does search work ?

Firsty, Let's understand how you'd search for a word in a given corpus of documents. Let's start with the naive way:

Given the below documents, say the word to search is "cricket" :

The naive approach would be to iterate over all the documents, and all the words inside each documents, and then add the matched result to the final answer.

As we can see the problem here is that to answer just one query we are touching every document , regardless of whether the word appears in one document or across all documents. And every query has to do this from scratch, and when there are million of records or documents, then this linear search becomes slow.

The naive approach isnt wrong, infact it is used when the documents are less , or when the search is required just once.

So this brings us to the data structure used in searches : Inverted Indexes

Inverted Index

In inverted index, the basic idea is to make text search fast so instead of storing the "documents -> words it contains" , we try to reverse/invert it to "words -> documents that contains it. "

For example , The inverted index for the 3 documents we considered for the naive approach would be :

The list of documents are called as posting lists.

Now a search for "cricket" is just a single dictionary lookup that immediately returns Docs 1 and 2. Searching "cricket stadium" intersects the two postings lists ([1,2] ? [2] = [2]), so doc 2 would be the only match.

Optimizations in Inverted Index

Now that we learned about the data structure, lets optimize a bit more on top of that, for faster, efficient, and informative lookups :

Stop Words Removal

Words like "a", "the", "an", "in", "on" appear in almost every document. So if we store these words in the

dictionary then the posting list would grow to contain all the documents, which just takes up memory but are contributing very less for the searches. So most search engines just drop these low information, high frequency words before putting them into the data structure.

Stemming

Next problem is “cricket” , “cricketer”, “cricketing” are all different words but if the user types “cricket” we would essentially want to show them all the documents containing “cricketer”, “cricketing” right.

So Stemming is basically stripping down the word to its root form by removing the suffix after applying some rule based transformations. Another example :

Lemmatization

Lemmatization does roughly the same job as stemming, reducing words to a base form, but it's smarter about it. Instead of blindly chopping suffixes, it uses an actual vocabulary and grammar knowledge to find the true dictionary form of a word.

Richer Posting Lists

So far our postings lists had just a list of document IDs. But we can pack in a lot more metadata information to add additional capabilities in search :

1. Term Frequency (tf) :

How many times the term occurs in the doc

> This information is useful for ranking search results. A document in which the search term appears multiple times may be considered more relevant than one in which it appears only once.

1. Positions :

Where in the doc does the term appear

Positions are useful for phrase searches and proximity searches.

For example, while searching for:

the search engine checks whether the position of Agents immediately follows the position of AI. This helps in u know much accurate searches.

1. Offsets :

Gives the character positioning

Offsets are mainly useful for highlighting matched words in search results and displaying relevant text snippets. For better viewer experience like you can see below as to how substack search does.

Q learn

X

POSTS



Zero to One : **Learning** Agents and Agentic Patterns

As I **learn** I have been documenting everything from various references, an... · Jan 21



The Power of Bloom Filters in System Design

...ding this newsletter, I am writing as I **learn** and explore these topics in de... · Jan 31



Concurrency Controls : Pessimistic vs Optimistic

...ding this newsletter, I am writing as I **learn** and explore these topics in de... · Mar 22



A Simple Proof of the CAP Theorem

...ding this newsletter, I am writing as I **learn** too. · Jan 4



How databases store data on the disk

...ding this newsletter, I am writing as I **learn** and explore these topics in de... · Feb 19



Apache Kafka Basics

...ding this newsletter, I am writing as I **learn** and explore these topics in de... · Feb 9



Caching Playbook for System Design Interviews

...ding this newsletter, I am writing as I **learn** too. · Jan 10



Client-Side Proxy vs Server-Side Proxy

...ding this newsletter, I am writing as I **learn** and explore these topics in de... · Jan 25



Everything you need to know about production ready RAG systems : Par...

...out them here - <https://www.pinecone.io/learn> · Nov 28

Champion Lists

A champion list stores only the top-ranked documents for each term instead of checking the entire posting list during every search.

For example:

The complete posting list may contain thousands of documents. A champion list keeps only the best few:

These documents are usually selected based on factors such as:

- High term frequency
- Importance of the term within the document
- Overall document quality or popularity

Keeping Posting Lists Sorted

Posting lists are usually stored in ascending order of doc_id.

Suppose a user searches for:

The search engine must find the document IDs that occur in both posting lists right. Lets see how it works with both unsorted and sorted lists.

Without sorted posting lists

When the lists are unsorted, the search engine may compare every document in the first list with every document in the second list.

If list1 contains m documents and list2 contains n documents, the time complexity is:

This becomes expensive when the posting lists contain thousands or millions of document IDs.

With sorted posting lists

When both lists are sorted, the search engine can use two pointers and move through both lists from left to right.

For example:

The result is:

The time complexity is:

Therefore, keeping posting lists sorted makes queries containing more than 1 word much faster

Posting List Compression

Posting lists may contain millions of document IDs, so storing every ID as a fixed-size integer can consume significant disk space and memory.

For example:

A common compression method is Time-series delta encoding, where we store the first document ID and then the gaps between consecutive IDs:

This works because posting lists are sorted. The original IDs can be reconstructed through cumulative addition.

Well, Delta encoding alone does not reduce space if every value is still stored as a fixed-size integer. Hence, it is combined with VarInts.

A VarInt (variable length integer encoding) stores small numbers using fewer bytes and larger numbers using more bytes (). For example:

So the original list may require about 10 bytes with VarInts, while the delta-encoded list may require only 6 bytes.

Simple illustration of how inverted indexes work, kudos to <https://x.com/BenjDicken>

[Inverted index.mov \(video — see the online note\)](#)



Time-series delta encoding

2026-02-10 · distributed systems

Source: Engineering notes — pradyumnac26.github.io

Time-series databases compress data using a simple idea: delta encoding. Most TS data has low variance (consecutive values are usually close together.) That predictability is what makes compression so effective.

Delta encoding for values

Instead of storing the full value each time, store the difference from the previous point.

Example: CPU usage recorded every second:

Raw values:

Delta encoded:

First value is stored as is; the rest are deltas.

Varints for smaller storage

Those deltas are usually tiny. Many time-series engines store them as varints (variable-length integers) : small deltas take fewer bytes than fixed 32/64-bit numbers. Storage becomes cheaper.

Trade-off: You use a bit of compute to reconstruct the values when reading.

Delta-of-delta for timestamps

Time-series databases do something extra for timestamps: delta-of-delta encoding.

When timestamps come at regular intervals (e.g., every 5 seconds):

Raw timestamps:

First-order deltas:

Delta-of-deltas:

When intervals are consistent, the second-order deltas are mostly zeros and zeros compress extremely well.

Simple concept. Huge impact on storage.

Rebuilding values from deltas is just repeated addition ($\text{value}[i] = \text{value}[i-1] + \text{delta}[i]$), and delta-of-delta just does that twice instead of once. Both are cheap, fixed-cost, predictable work.

The real cost is in decoding the varints. Because a varint's byte-length varies per value, the decoder can't know

where one number ends and the next begins until it reads a continuation bit byte-by-byte, unlike fixed-width deltas, where every value's position is known in advance. That serial, unpredictable byte-walk is what actually costs CPU cycles on read, not the addition used to undo the delta encoding itself.

Database Indexing Best Practises

2026-02-26 · *distributed systems*

Source: Engineering notes — pradyumnac26.github.io

A bad database index is worse than no index.

Database indexing can be expensive if not done the right way.

here are some best practices to follow :

- add indexes based on actual query patterns, especially columns used in filters, joins, and sorting
- use explain analyze before adding an index to understand the query path, and after adding it to confirm the database is using the right index properly.
- indexes can slow down writes, because every insert, update, and delete may now need to change in both the original table and the related indexes table. So avoid adding unnecessary indexes.
- avoid indexing huge fields like large text columns, because that can make the index bulky, slower, and more expensive to maintain